SCHOOL OF NATURAL AND APPLIED SCIENCES

DEPARTMENT OF COMPUTING

TO : MR. S. YUTE

FROM : MOSES SAMUEL & CHISOMO KAWINJO

REGISTRATION NO : BSC-COM-NE-33-23 & BSC-COM-NE-23-33

COURSE NAME : NETWORK PROGRAMMING AND APPLICATION DEVELOPMENT

COURSE CODE : NET322

ASSIGNMENT TITLE : IoT TELEMETRY PIPELINE & DASHBOARD

DUE DATE : 9TH APRIL, 2026

Introduction

This document outlines the planning and architectural design of an IoT Telemetry Pipeline and Dashboard system that simulates greenhouse monitoring using asynchronous Python networking. The system consists of multiple simulated sensors that send telemetry data to a central AsyncIO server over TCP using Protocol Buffers. The server stores the data in SQLite, provides a REST API for historical access, and streams real-time updates to dashboard clients via WebSockets. The architecture focuses on asynchronous I/O, efficient data serialization, real-time communication, and flexible REST responses using JSON, XML, and YAML through content negotiation.

Architecture Diagram

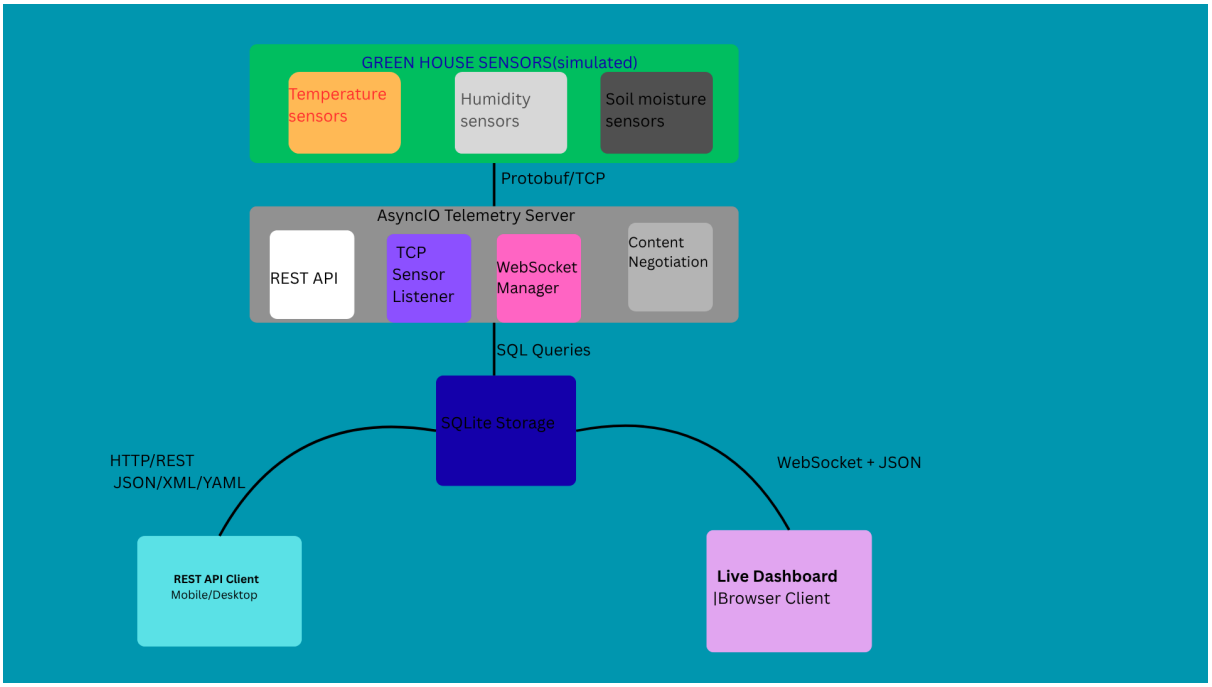

Figure 1: System Architecture Diagram

Architecture Explanation

The system uses a centralized client-server architecture where asynchronous sensor simulators generate greenhouse telemetry data such as temperature, humidity, soil moisture, and light intensity. Sensors transmit readings to an AsyncIO telemetry server using Protocol Buffers over TCP for efficient binary serialization and reduced bandwidth usage. The server concurrently manages sensor connections, REST API requests, and WebSocket sessions using Python coroutines and the AsyncIO event loop without threads. Telemetry data is stored in SQLite for historical queries, while REST clients access data in JSON, XML, or YAML through HTTP content negotiation, and dashboard clients receive live updates through WebSocket connections. .

Sequence Diagram

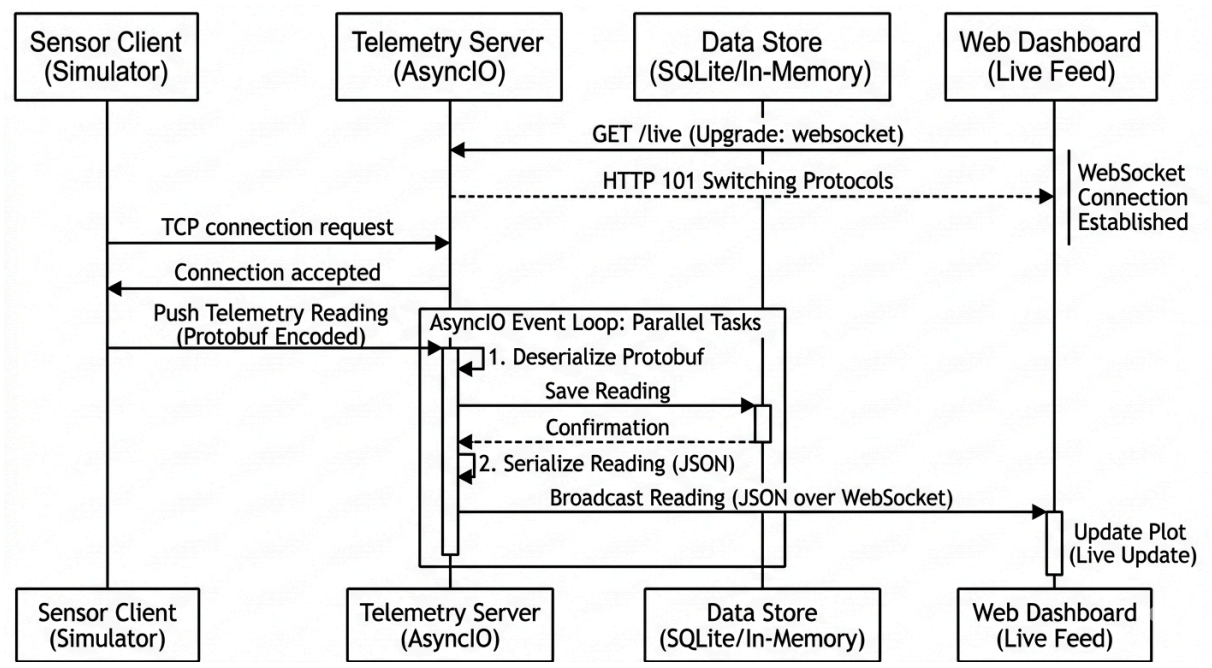

Figure 2: Sensor-to-Dashboard Sequence Flow

Sequence Explanation

The telemetry process begins when a dashboard client establishes a WebSocket connection to the server through the /live endpoint using the HTTP WebSocket upgrade handshake. Sensor simulators asynchronously generate telemetry readings, serialize them using Protocol Buffers, and transmit them to the server through TCP sockets. The AsyncIO telemetry server deserializes the binary payload into structured sensor data and stores the readings in an SQLite database for historical access. After storage, the server broadcasts live telemetry updates to connected WebSocket clients in JSON format, allowing dashboard clients to display sensor readings in real time.

DATA MODEL

Sensor Entity

The Sensor entity represents a greenhouse telemetry device registered within the system. Each sensor maintains an identifier and reporting configuration.

Field	Type	Description
id	string	Unique sensor identifier
interval	integer	Reporting interval in seconds

Reading Entity

The Reading entity stores telemetry data generated by sensors and received by the telemetry server.

Field	Type	Description
id	integer	Primary key
sensor_id	string	Sensor identifier
temperature	float	Temperature value
humidity	float	Humidity percentage
light	float	Light intensity
timestamp	integer	Unix timestamp
soil_moisture	float	Soil moisture percentage

Protobuf Schema

```
syntax = "proto3";

message SensorReading {
  string sensor_id = 1;
  float temperature = 2;
  float humidity = 3;
  float soil_moisture = 4;
  float light = 5;
  int64 timestamp = 6;
}
```

REST API SPECIFICATION

GET /sensors

Property	Value
Method	GET
Path	/sensors

Purpose	Retrieve registered sensors
Request Body	None
Response	JSON/XML/YAML sensor list
Status Codes	200 OK

Example:

```
[
  {
    "sensor_id": "greenhouse-1"
  }
]
```

GET /sensors/{id}/readings

Property	Value
Method	GET
Path	/sensors/{id}/readings
Purpose	Retrieve historical readings
Request Body	None
Response	Telemetry history
Status Codes	200, 404

Example:

```
[
 {
   "sensor_id": "greenhouse-1",
   "temperature": 27.4
 }
]
```

POST /sensors

Property	Value
Method	POST

Path	/sensors
Purpose	Register sensor
Request Body	Sensor metadata
Response	Success message
Status Codes	201, 400

Example

```
{
 "id": "greenhouse-4"
}
```

DELETE /sensors/{id}

Properties	**Value**
Method	DELETE
Path	/sensors/{id}
Purpose	Remove sensor
Request body	None
Response	Confirmation message
Status Codes	200, 404

YAML CONFIGURATION SCHEMA

The sensor simulator loads runtime configuration from a YAML configuration file. YAML was selected because it is human-readable and easy to modify without changing source code

sensors:

```yaml
- id: greenhouse-1
  interval: 3

- id: greenhouse-2
  interval: 5

- id: greenhouse-3
  interval: 7
```

Field	Description
id	Unique sensor identifier
interval	Reporting interval in seconds

Concurrency Model

The system uses Python AsyncIO to achieve asynchronous concurrency through coroutines and the event loop without relying on threads. The telemetry server concurrently handles TCP sensor connections, REST API requests, and WebSocket live-feed sessions using non-blocking I/O operations for improved scalability and efficiency. Telemetry readings are processed asynchronously and stored using aiosqlite to prevent blocking the event loop during database operations. Live telemetry updates are then broadcast to connected WebSocket clients in real time. If a client becomes a slow consumer and cannot process updates quickly enough, the server removes the stalled connection to prevent blocking and excessive memory usage.

Failure Handling

The system includes basic fault tolerance to maintain reliability during runtime failures. If a sensor disconnects unexpectedly, the server detects the closed TCP connection and safely terminates the related coroutine without affecting other sensors, while simulators automatically attempt reconnection. Malformed Protocol Buffer data is handled gracefully by catching deserialization errors, logging the issue, and continuing normal operation. If the SQLite database becomes unavailable or write operations fail, the error is logged and the system continues serving active connections, though HTTP endpoints may return a 500 error for failed storage actions. During high load, AsyncIO manages concurrent connections via the event loop, but slow clients may be disconnected to prevent resource exhaustion, with future improvements planned such as rate limiting and queue management.

CONCLUSION

The proposed IoT telemetry system demonstrates the use of asynchronous network programming for real-time greenhouse monitoring. The architecture combines AsyncIO, Protocol Buffers, REST APIs, WebSockets, and SQLite to provide efficient telemetry processing and storage. AsyncIO enables the server to handle multiple sensor connections concurrently without using threads. Protocol Buffers provide compact and fast binary serialization for sensor communication. REST APIs with content negotiation allow clients to

retrieve data in JSON, XML, or YAML formats. WebSocket communication enables live telemetry updates for dashboard clients in real time. Overall, the system satisfies the assignment requirements while demonstrating modern IoT and network programming concepts.